

UNIVERSIDAD DA VINCI DE GUATEMALA

Facultad de Ingeniería en Sistemas y Ciencias de la Computación
Curso: Estructuras de Datos

FICHAS TÉCNICAS

Estructuras Arbóreas de Datos

eficiencia y aplicaciones de los árboles de datos más utilizados en la industria
profunda sobre el funcionamiento, eficiencia y aplicaciones reales de los
información en un formato profesional de ficha técnica.

BST	Árbol Binario de Búsqueda	AVL	Árbol Balanceado AVL	B/B+	Árbol B+ B+	Trie	Trie (Árbol de Prefijos)	Heap	Heap (Montículo)
-----	---------------------------	-----	----------------------	------	-------------	------	--------------------------	------	------------------

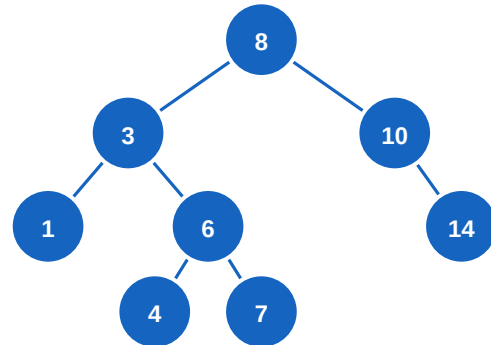
FASE 1 — Fichas Técnicas	50% de la nota	5 fichas técnicas (una por estructura). Máx. 1 página por árbol.
FASE 2 — Implementación Práctica	50% de la nota	BST en Java + defensa en video (máx. 5 min) subido a YouTube.

Nombre	del	Estudiante:	Carné: _____
Fecha	de	Entrega:	Sección: _____

■ IDENTIFICACIÓN Y DEFINICIÓN

Un Árbol Binario de Búsqueda (BST) es una estructura de datos jerárquica en la que cada nodo tiene como máximo dos hijos: el hijo izquierdo contiene únicamente valores menores que el nodo padre, y el hijo derecho contiene únicamente valores mayores. Esta propiedad se mantiene recursivamente en todos los sub-árboles, lo que permite realizar búsquedas, inserciones y eliminaciones de forma eficiente.

■ VISUALIZACIÓN



Σ ANÁLISIS DE COMPLEJIDAD ALGORÍTMICA (Big O)

Operación	Peor Caso	Caso Promedio	Observación
Búsqueda	$O(n)$	$O(\log n)$	$O(n)$ cuando el árbol está completamente desbalanceado
Inserción	$O(n)$	$O(\log n)$	Degrada a lista enlazada con datos ya ordenados
Eliminación	$O(n)$	$O(\log n)$	Requiere encontrar sucesor in-order en nodo de 2 hijos
Espacio	$O(n)$	$O(n)$	Un nodo por cada elemento almacenado

■ CARACTERÍSTICAS MECÁNICAS

Organización: Cada nodo almacena un valor y punteros al hijo izquierdo y derecho. La propiedad BST exige: *izquierdo* < *raíz* < *derecho* en cada sub-árbol. **Búsqueda:** Compara el valor buscado con el nodo actual; si es menor va izquierda, si es mayor va derecha, recursivamente hasta encontrarlo o llegar a null. **Eliminación:** Tres casos: (1) nodo hoja → se elimina directamente; (2) un hijo → se reemplaza por ese hijo; (3) dos hijos → se reemplaza con el sucesor in-order (mínimo del sub-árbol derecho).

■ VENTAJA PRINCIPAL

Búsqueda eficiente en $O(\log n)$ promedio; implementación simple; in-order traversal produce la secuencia ordenada; útil cuando los datos son dinámicos (inserciones y eliminaciones frecuentes).

■ LIMITACIÓN PRINCIPAL

Sin balanceo automático puede degenerar a $O(n)$ en el peor caso (datos ya ordenados). No garantiza rendimiento constante sin variantes auto-balanceadas (AVL, Red-Black).

■ CASO DE USO EN LA INDUSTRIA

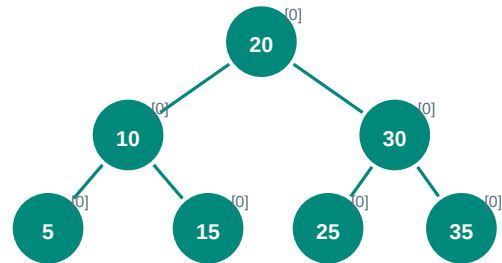
Software/Sistema real: Git (índice de objetos) · C++ STL (`std::set` / `std::map`) · SQLite (índices B-tree sobre BST conceptual)

Implementación interna de los módulos `std::set` y `std::map` en C++ STL (usando Red-Black Trees, variante del BST). También en **sistemas de indexación** de bases de datos relacionales para búsquedas por rango. Git usa un BST para indexar objetos en el sistema de archivos.

■ IDENTIFICACIÓN Y DEFINICIÓN

Un Árbol AVL es un Árbol Binario de Búsqueda con auto-balanceo. Para cada nodo, la diferencia de altura entre su sub-árbol izquierdo y derecho (factor de balance) nunca excede $|1|$. Cuando una inserción o eliminación viola esta condición, el árbol realiza rotaciones (simple o doble) para restaurar el balance, garantizando rendimiento $O(\log n)$ en todas las operaciones.

■ VISUALIZACIÓN



Σ ANÁLISIS DE COMPLEJIDAD ALGORÍTMICA (Big O)

Operación	Peor Caso	Caso Promedio	Observación
Búsqueda	$O(\log n)$	$O(\log n)$	Garantizado por el balanceo estricto
Inserción	$O(\log n)$	$O(\log n)$	Incluye $O(1)$ rotaciones como máximo
Eliminación	$O(\log n)$	$O(\log n)$	Puede requerir múltiples rotaciones
Espacio	$O(n)$	$O(n)$	Factor de balance por nodo (+1 campo extra)

■■ CARACTERÍSTICAS MECÁNICAS

Factor de Balance (FB): $FB = \text{altura(derecho)} - \text{altura(izquierdo)}$. Debe estar en $\{-1, 0, +1\}$. **Rotación Simple Derecha (RD):** Se aplica cuando $FB = -2$ en un nodo y $FB = -1$ en su hijo izquierdo. **Rotación Simple Izquierda (RI):** Caso espejo ($FB = +2$, hijo derecho $FB = +1$). **Rotación Doble (LD/LI):** Cuando los factores de balance tienen signos opuestos, se combinan dos rotaciones simples. El árbol recalcula alturas bottom-up después de cada operación.

■ VENTAJA PRINCIPAL

Garantiza $O(\log n)$ en el peor caso para todas las operaciones. Óptimo para aplicaciones con muchas búsquedas (lectura intensiva). Más estrictamente balanceado que Red-Black Trees.

■■ LIMITACIÓN PRINCIPAL

Las rotaciones hacen que la inserción y eliminación sean más lentas que en un BST simple. El almacenamiento del factor de balance agrega overhead por nodo. Complejidad de implementación mayor que BST.

■ CASO DE USO EN LA INDUSTRIA

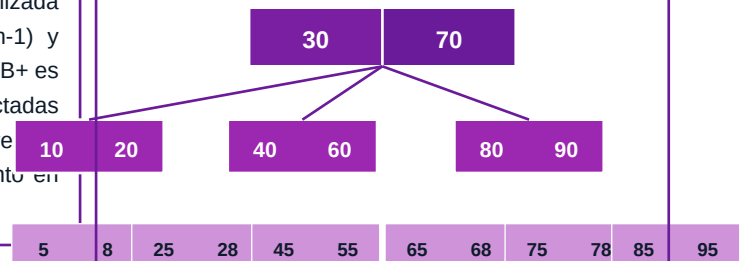
Software/Sistema real: Linux Kernel (gestión de memoria virtual) · Sistemas de trading HFT · Compiladores (tablas de símbolos)

Usado en **bases de datos en memoria** que requieren búsquedas ultra-rápidas. **Linux Kernel** usa variantes para gestionar memoria virtual (mm_struct). **Java TreeMap** usa Red-Black (prima del AVL) para el mismo propósito. Sistemas de trading de alta frecuencia usan AVL para order books.

■ IDENTIFICACIÓN Y DEFINICIÓN

Un Árbol B es una estructura de datos auto-balanceada generalizada donde cada nodo puede contener múltiples claves (hasta $m-1$) y múltiples hijos (hasta m), donde m es el orden del árbol. El Árbol B+ es una variante donde todos los datos residen en las hojas (conectadas como lista enlazada) y los nodos internos solo almacenan clave enrutamiento. Está optimizado para sistemas de almacenamiento en disco.

■ VISUALIZACIÓN



Σ ANÁLISIS DE COMPLEJIDAD ALGORÍTMICA (Big O)

Operación	Peor Caso	Caso Promedio	Observación
Búsqueda	$O(\log n)$	$O(\log n)$	Altura del árbol proporcional a $\log_m(n)$
Inserción	$O(\log n)$	$O(\log n)$	Incluye splits de nodos cuando están llenos
Eliminación	$O(\log n)$	$O(\log n)$	Puede requerir merge o redistribución de nodos
Espacio	$O(n)$	$O(n)$	Factor de ramificación m reduce altura drásticamente

■ CARACTERÍSTICAS MECÁNICAS

Propiedades clave: Todas las hojas están al mismo nivel (árbol perfectamente balanceado en altura). Cada nodo interno tiene entre $\lceil m/2 \rceil$ y m hijos. **Split:** Cuando un nodo se llena ($m-1$ claves), se divide en dos y la clave media sube al padre. **B+ vs B:** En B+, los nodos internos no almacenan datos, solo claves para navegación. Las hojas de B+ están enlazadas en lista, permitiendo recorridos secuenciales eficientes (ideal para queries de rango en SQL). El factor de ramificación alto (típicamente 100-1000) minimiza accesos a disco.

■ VENTAJA PRINCIPAL

Minimiza accesos a disco al maximizar datos por nodo (factor de ramificación alto). B+ permite scans de rango eficientes ($O(k)$ para k resultados). Balanceo automático garantizado. Óptimo para datos que no caben en RAM.

■ LIMITACIÓN PRINCIPAL

Implementación compleja (splits y merges). Overhead de espacio en nodos no-hoja de B+. Menos eficiente que BST/AVL para datos completamente en memoria.

■ CASO DE USO EN LA INDUSTRIA

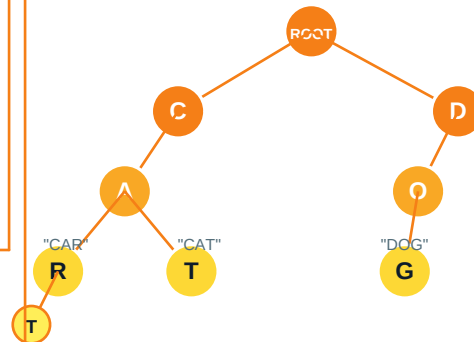
Software/Sistema real: MySQL InnoDB (índices primarios y secundarios) · PostgreSQL (índices B-tree por defecto) · NTFS / ext4 / HFS+ (sistemas de archivos)

Es la estructura fundamental de los **motores de bases de datos** relacionales. **MySQL InnoDB** usa B+ trees para todos sus índices. **PostgreSQL** usa B-trees por defecto. **Sistemas de archivos** como NTFS, ext4 y HFS+ usan B-trees para sus estructuras de directorio.

■ IDENTIFICACIÓN Y DEFINICIÓN

Un Trie (del inglés *retrieval*) es una estructura de árbol especializada para almacenar cadenas de texto, donde cada nodo representa un carácter de la cadena. Las claves no se almacenan en los nodos sino que se definen por el camino desde la raíz hasta ese nodo. Todos los descendientes de un nodo comparten el mismo prefijo, lo que hace al Trie extremadamente eficiente para operaciones de búsqueda por prefijo.

■ VISUALIZACIÓN



Σ ANÁLISIS DE COMPLEJIDAD ALGORÍTMICA (Big O)

Operación	Peor Caso	Caso Promedio	Observación
Búsqueda	$O(m)$	$O(m)$	m = longitud de la clave buscada, independiente de n
Inserción	$O(m)$	$O(m)$	Crea máximo m nodos nuevos por inserción
Eliminación	$O(m)$	$O(m)$	Requiere limpieza de nodos huérfanos
Espacio	$O(n \cdot m)$	$O(n \cdot m)$	n = número de claves, m = longitud promedio

■ CARACTERÍSTICAS MECÁNICAS

Estructura: La raíz es un nodo vacío. Cada nivel representa una posición de carácter. Cada nodo tiene hasta 26 hijos (para alfabeto inglés) o 256 (ASCII completo). Un nodo marcado como *end_of_word* indica que el camino hasta ese nodo forma una palabra válida. **Búsqueda por prefijo:** Se navega por los caracteres del prefijo; si se llega al último carácter sin errores, todos los descendientes son palabras con ese prefijo. **Trie comprimido (PATRICIA):** Comprime cadenas de nodos de un solo hijo para reducir espacio.

■ VENTAJA PRINCIPAL

Búsqueda en $O(m)$ independiente del número de claves n . Autocompletado y búsqueda por prefijo son naturales. No requiere función hash, sin colisiones. Ordena strings lexicográficamente de forma implícita.

■ LIMITACIÓN PRINCIPAL

Alto consumo de memoria (hasta 26 punteros por nodo aunque estén vacíos). Ineficiente para vocabularios con pocas palabras largas. No es adecuado para claves numéricas o datos no-textuales.

■ CASO DE USO EN LA INDUSTRIA

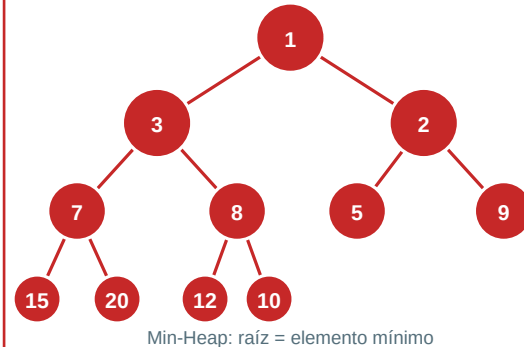
Software/Sistema real: Google Search (autocompletado de búsquedas) · iOS/Android (teclado predictivo) · Apache Lucene (indexación de texto)

El **autocompletado** de Google, Bing y los teclados inteligentes de iOS/Android usa Tries. Los **motores de búsqueda** los usan para indexación de texto. **Spell-checkers** como los de Microsoft Word y herramientas como **grep** usan variantes de Trie para matching eficiente.

■ IDENTIFICACIÓN Y DEFINICIÓN

Un Heap (montículo) es un árbol binario completo que satisface la propiedad de montículo: en un **Max-Heap**, el valor de cada nodo es mayor o igual al de sus hijos (la raíz es el máximo). En un **Min-Heap**, el valor de cada nodo es menor o igual al de sus hijos (la raíz es el mínimo). Se implementa eficientemente como un arreglo (array), donde para el nodo en índice i : hijo izquierdo = $2i+1$, hijo derecho = $2i+2$.

■ VISUALIZACIÓN



Σ ANÁLISIS DE COMPLEJIDAD ALGORÍTMICA (Big O)

Operación	Peor Caso	Caso Promedio	Observación
Inserción	$O(\log n)$	$O(\log n)$	Se inserta al final y hace sift-up
Extraer Min/Max	$O(\log n)$	$O(\log n)$	Reemplaza raíz con último nodo y sift-down
Peek (ver raíz)	$O(1)$	$O(1)$	La raíz siempre es el mínimo/máximo
Build Heap	$O(n)$	$O(n)$	Heapify bottom-up más eficiente que n inserciones
Heap Sort	$O(n \log n)$	$O(n \log n)$	In-place, sin espacio extra

■■ CARACTERÍSTICAS MECÁNICAS

Representación en array: El árbol completo se almacena en un arreglo sin punteros. Para nodo en índice i : padre = $\lfloor (i-1)/2 \rfloor$, hijo izq = $2i+1$, hijo der = $2i+2$. **Sift-up (heapify up):** Tras insertar al final, el elemento sube intercambiándose con su padre mientras viole la propiedad heap. **Sift-down (heapify down):** Tras extraer la raíz, el último elemento va a la raíz y baja intercambiándose con el hijo de mayor/menor valor. **Build Heap:** Aplicar sift-down a todos los nodos internos de derecha a izquierda es $O(n)$.

■ VENTAJA PRINCIPAL

Acceso al mínimo o máximo en $O(1)$. Inserción y extracción en $O(\log n)$. Implementación como array es cache-friendly. Heap Sort es in-place. Base natural de las colas de prioridad.

■■ LIMITACIÓN PRINCIPAL

Búsqueda de un elemento arbitrario es $O(n)$ (no tiene propiedad BST). No soporta búsqueda por clave eficientemente. No adecuado para rangos. Acceso solo al extremo (min o max).

■ CASO DE USO EN LA INDUSTRIA

Software/Sistema real: Google Maps (Dijkstra con Min-Heap) · Linux Kernel (scheduler CFS) · Compresión ZIP/GZIP (Huffman coding)

El **algoritmo de Dijkstra** (GPS, Google Maps, redes) usa Min-Heap para la cola de prioridad. Los **sistemas operativos** (Linux, Windows) usan heaps para el scheduler de procesos. **Huffman Coding** (compresión ZIP, JPEG, MP3) usa Min-Heap para construir el árbol de codificación.

FASE 2 — IMPLEMENTACIÓN PRÁCTICA

Árbol Binario de Búsqueda (BST) en Java + Defensa en Video

1. REQUISITOS DEL CÓDIGO (JAVA)

Método Requerido	Descripción de Implementación
<code>insert(int valor)</code>	Coloca el nodo en la posición correcta manteniendo las reglas BST (izq < raíz < der)
<code>search(int valor)</code>	Retorna boolean: true si el valor existe en el árbol, false en caso contrario
<code>delete(int valor)</code>	Elimina el nodo considerando los 3 casos: hoja, un hijo, dos hijos (usar sucesor in-order)
recorrido InOrder	Imprime nodos en orden ascendente (izquierdo → raíz → derecho)
recorrido PreOrder	Imprime raíz primero (raíz → izquierdo → derecho) — útil para copiar árbol
recorrido PostOrder	Imprime raíz al final (izquierdo → derecho → raíz) — útil para liberar memoria

2. REQUISITOS DE LA DEFENSA EN VIDEO (MÁX. 5 MINUTOS)

Segmento	Qué debe demostrar
■ Ejecución en Vivo	Mostrar el programa corriendo. Insertar al menos 5 valores, eliminar al menos 2, y ejecutar los 3 recorridos mostrando la salida en consola.
Explicación del Código	Explicar la lógica del método <code>delete()</code> — el más complejo. Mencionar los 3 casos de eliminación y cómo se encuentra el sucesor in-order.
■ Sustentación Teórica	Justificar verbalmente la complejidad Big-O del método <code>search()</code> : por qué es $O(\log n)$ en promedio y $O(n)$ en el peor caso.

3. RÚBRICA DE CALIFICACIÓN (100%)

Precisión Técnica	Fase 1	25%	Datos correctos de complejidad Big-O y comprensión de las reglas de cada estructura.
Presentación Visual	Fase 1	25%	Claridad visual, ortografía correcta, uso de diagramas claros y entrega en PDF.
Funcionalidad del Código	Fase 2	25%	Correcta implementación de <code>insert</code> , <code>search</code> , <code>delete</code> (3 casos) y los 3 recorridos.
Defensa en Video	Fase 2	25%	Claridad en la explicación, demostración de ejecución y justificación de complejidad.